

ASP.NET Core MVC - Routing

Routing in ASP.NET Core

Routing is the "GPS" of your application. It maps a URL to a specific Controller Action.

Overview of Routing

Routing is the process of directing incoming HTTP requests to the appropriate controller and action method in a web application.

In simple terms:

Routing tells the application **which controller and action should handle a specific URL request**.

Example

When a user enters:

`https://example.com/Home/Index`

Routing determines:

- Controller → `HomeController`
- Action → `Index()`
- Response → Display the Index page

So routing acts like a **traffic manager** that decides where requests should go.

Using the MapRoute Method to Configure Routing

What is MapRoute?

`MapRoute` is used to define **URL patterns** and connect them to controllers and actions.

It is commonly used in **conventional routing** to define how URLs should be structured.

Basic Syntax

```
routes.MapRoute(  
    name: "Default",  
    url: "{controller}/{action}/{id}",  
    defaults: new { controller = "Home", action = "Index", id = UrlParameter.Optional }  
);
```

Explanation

name

This is the name of the route.

"Default"

Used for identifying the route.

url

Defines the URL structure.

```
{controller}/{action}/{id}
```

This means:

```
example.com/Home/Index/1
```

Where:

Part	Meaning
controller	Home
action	Index
id	1

defaults

Provides default values.

```
new { controller = "Home", action = "Index", id = UrlParameter.Optional }
```

Meaning:

If no controller or action is provided:

```
example.com/
```

It automatically goes to:

HomeController → Index()

Example

URL

```
https://example.com/Product/Details/5
```

Controller

```
public class ProductController : Controller
{
    public ActionResult Details(int id)
    {
```

```
return View();  
}  
}
```

Result

Routing sends the request to:

ProductController → Details(5)

Different Types of Routing

There are **two main types of routing**:

1. Conventional Routing
 2. Attribute-Based Routing
-

1. Conventional Routing

Definition

Conventional routing defines routing rules in a **central location** like:

RouteConfig.cs

Program.cs

Startup.cs

or using `MapRoute` .

It follows a **pattern-based approach**.

Example

```
routes.MapRoute(  
    name: "StudentRoute",  
    url: "Student/{action}/{id}",  
    defaults: new { controller = "Student", action = "Index", id = UrlParameter.Optional }  
);
```

URL

```
example.com/Student/Details/10
```

Controller

```
public class StudentController : Controller  
{  
    public ActionResult Details(int id)  
    {  
        return View();  
    }  
}
```

```
}  
}
```

Convention Used

URL → Controller → Action

Pattern:

Student/{action}/{id}

Advantages

- ✓ Easy to manage
- ✓ Centralized configuration
- ✓ Good for large applications
- ✓ Simple to understand

Disadvantages

- ✗ Less flexible
- ✗ All routes must follow the defined pattern

2. Attribute-Based Routing

Definition

Attribute-based routing defines routing **directly inside the controller using attributes**.

Instead of defining routes in `MapRoute`, you place routes on actions.

Example

```
[Route("student/details/{id}")]  
public ActionResult Details(int id)  
{  
    return View();  
}
```

URL

example.com/student/details/5

Full Controller Example

```
public class StudentController : Controller
{
    [Route("student")]
    public ActionResult Index()
    {
        return View();
    }

    [Route("student/details/{id}")]
    public ActionResult Details(int id)
    {
        return View();
    }
}
```

Convention Used

Attribute → URL → Action

Routing is written directly on the action.

Advantages

- ✓ More flexible
 - ✓ Easy to understand routes
 - ✓ Cleaner code
 - ✓ Better control over URLs
-

Disadvantages

- ✗ Hard to manage in very large projects
 - ✗ Routes are spread across controllers
-

Conventional vs Attribute Routing

Feature	Conventional Routing	Attribute Routing
Location	RouteConfig / Program.cs	Controller
Control	Centralized	Distributed
Flexibility	Less flexible	More flexible
URL Definition	Pattern-based	Direct URL
Maintenance	Easy for large apps	Easy for small apps

Using HTTP Verbs in Attribute-Based Routing

What are HTTP Verbs?

HTTP verbs define **what action the user wants to perform**.

Common HTTP Verbs

Verb	Purpose
GET	Retrieve data
POST	Send data
PUT	Update data
DELETE	Remove data

GET Method

Used to retrieve data.

Example

```
[HttpGet]
[Route("student/list")]
public ActionResult GetStudents()
{
    return View();
}
```

URL

```
example.com/student/list
```

Purpose

Retrieve student list.

POST Method

Used to submit data.

Example

```
[HttpPost]
[Route("student/create")]
public ActionResult CreateStudent(Student s)
{
    return View();
}
```

Purpose

Add a new student.

PUT Method

Used to update data.

Example

```
[HttpPut]
[Route("student/update/{id}")]
public ActionResult UpdateStudent(int id)
{
    return View();
}
```

Purpose

Update student information.

DELETE Method

Used to remove data.

Example

```
[HttpDelete]
[Route("student/delete/{id}")]
public ActionResult DeleteStudent(int id)
{
    return View();
}
```

Purpose

Delete a student.

Combined Attribute Routing with HTTP Verbs

Example

```
[Route("student")]
public class StudentController : Controller
{
    [HttpGet]
    [Route("list")]
    public ActionResult List()
    {
        return View();
    }

    [HttpPost]
    [Route("create")]
    public ActionResult Create(Student s)
    {
        return View();
    }

    [HttpPut]
    [Route("update/{id}")]
```

```
public ActionResult Update(int id)
{
    return View();
}

[HttpDelete]
[Route("delete/{id}")]
public ActionResult Delete(int id)
{
    return View();
}
}
```

How Routing Works (Step-by-Step)

Step 1

User enters URL

example.com/student/details/5

Step 2

Routing system reads URL

Step 3

Find matching route

Step 4

Calls controller

StudentController

Step 5

Calls action

Details(5)

Step 6

Returns View

Real-World Example

Think of routing like **Google Maps** 

Routing	Google Maps
URL	Address
Controller	City
Action	Street
ID	House number
MapRoute	Route configuration

Routing	Google Maps
Attribute	Direct GPS
Routing	coordinates

Routing helps the request **reach the correct destination**.

Summary

Routing

- Directs URL requests to controllers and actions
- Works like a traffic manager

MapRoute

- Used in conventional routing
- Defines URL patterns
- Centralized configuration

Conventional Routing

- Defined in RouteConfig or Program.cs
- Pattern-based
- Easy for large applications

Attribute Routing

- Defined in controller
- Flexible and clean
- Direct URL mapping

HTTP Verbs

- GET → retrieve data
- POST → send data
- PUT → update data
- DELETE → remove data